



Loyiha: Flowgorithm'da "Kalkulyator"

Kalkulyator loyihasining maqsadi va ishlash mantig'ini rejalashtirish; foydalanuvchidan ikki son va amal turini kiritish; If bloklari orqali tanlangan amalni aniqlash; qo'shish, ayirish, ko'paytirish, bo'lish; nolga bo'lish va noto'g'ri amal xatolarini tekshirish; menyu sikli bilan takroriy hisoblash; test rejasi va loyiha himoyasi.

2 dars · amaliy

Bu loyihada nimani o'rganasiz

- Loyihani **rejalashtirish**: maqsad, kirish, chiqish, cheklovlar;
- Bir nechta If bilan foydalanuvchi **tanlovini** aniqlashni;
- **Xatolarni oldindan ko'rish**: nolga bo'lish, noto'g'ri amal;
- Kalkulyatorni sikl bilan **qayta-qayta** ishlaydigan qilishni;
- Loyihani **test qilish** va taqdim etishni.

Loyiha rejasi (2 dars)

1-dars: rejalashtirish, 1-versiya (4 amal) va 2-versiya (xatolarni tekshirish). **2-dars**: 3-versiya (menyu sikli), qo'shimcha amallar, test jadvali, GitHub'ga joylash va himoya.

1-bosqich: rejalashtirish

Professional dasturchilar kod yozishdan oldin **texnik topshiriq** tuzadi. Biz ham shunday qilamiz:

Savol	Javob
Maqsad	Ikki son ustida 4 ta arifmetik amalni bajaruvchi dastur

Savol	Javob
Kirish	<code>a</code> (Real), <code>b</code> (Real), <code>amal</code> (String: <code>+</code> <code>-</code> <code>*</code> <code>/</code>)
Chiqish	<code>a amal b = natija</code> , masalan <code>12 / 4 = 3</code>
Qayta ishlash	Amalga qarab mos formulani tanlash
Xatolar	<code>b = 0</code> bo'lganda bo'lish; noma'lum amal (<code>x</code> , <code>%</code> , bo'sh)

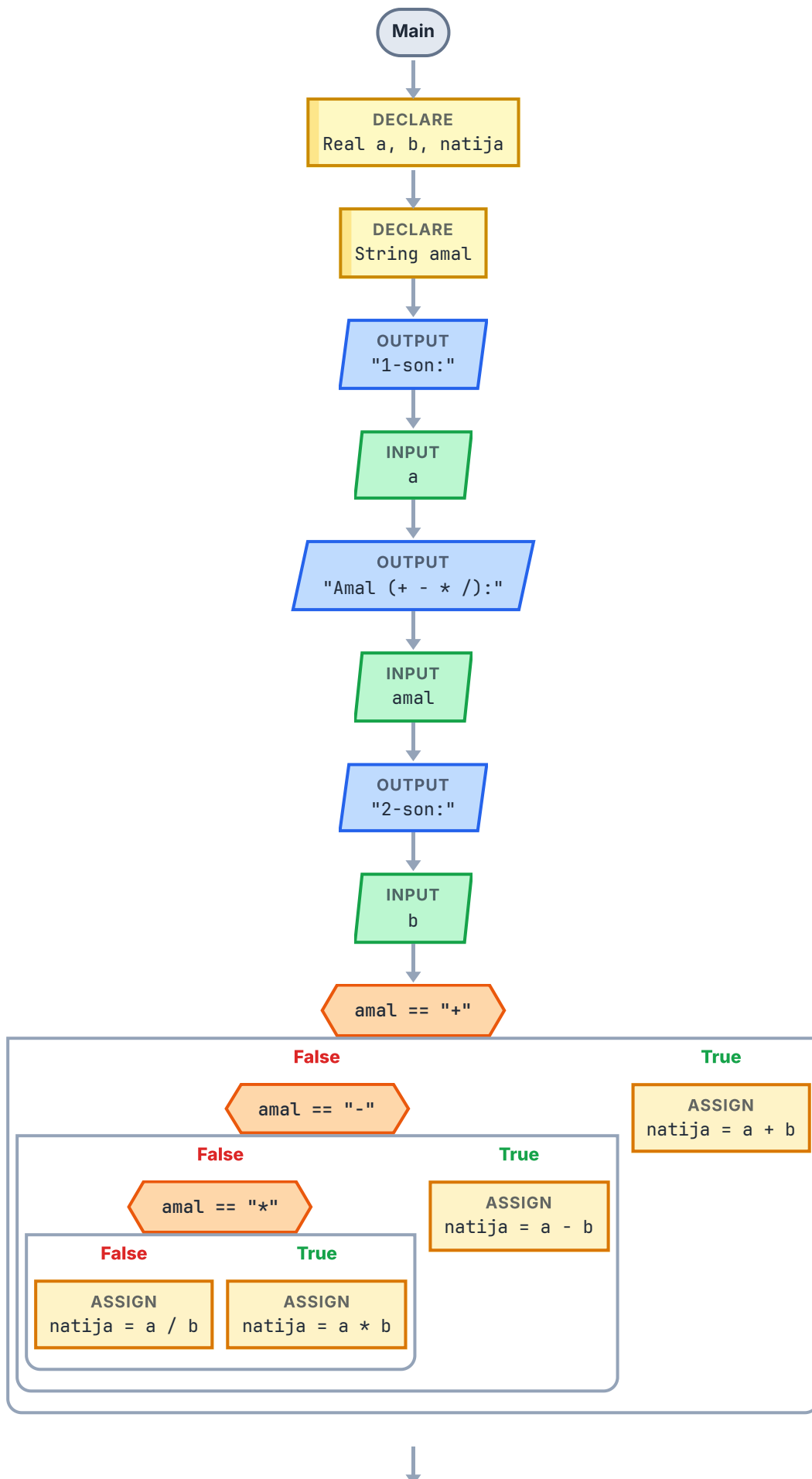


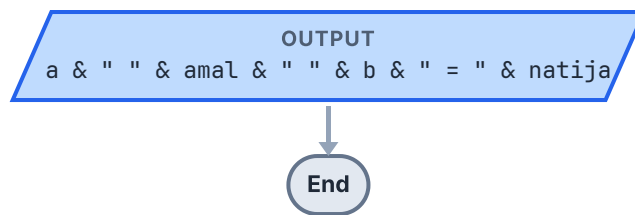
O'xshatish: kassir

Kassirga ikkita narsa beriladi — pul va buyurtma. U buyurtmaga qarab (choy, qahva, sharbat) **tegishli** ishni bajaradi. Menyuda yo'q narsani so'rasangiz — "Bunday mahsulot yo'q" deydi, jim turmaydi. Bizning kalkulyator ham shunday bo'lishi kerak.

2-bosqich: 1-versiya — to'rt amal

Amalni tanlash uchun "zinapoya" usulidagi If'lar (20-mavzu): `+` mi? Yo'q — `-` mi? Yo'q — `*` mi? ...

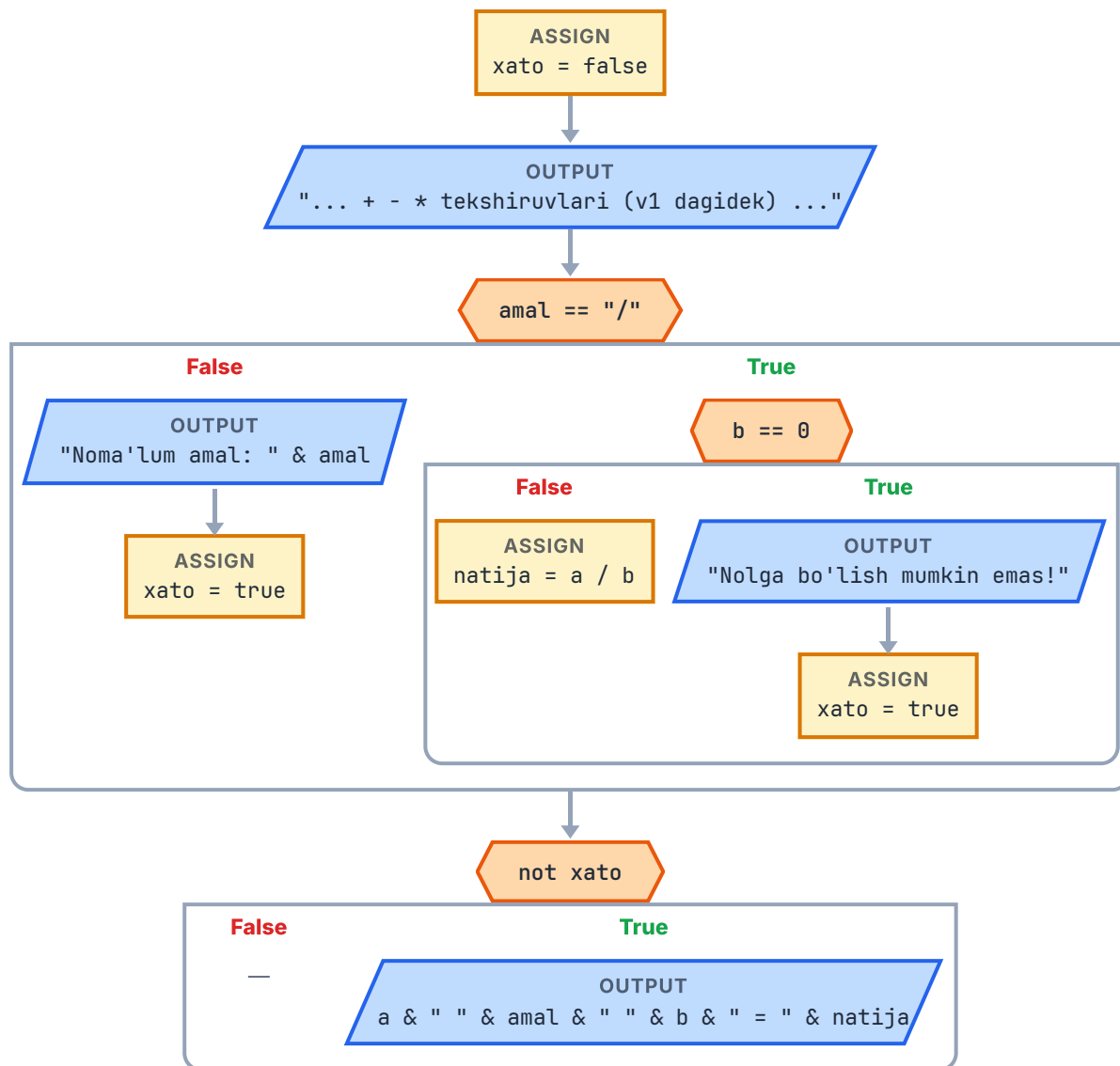




Muammolar: (1) `b = 0` va `/` — dastur xato bilan to'xtaydi. (2) Foydalanuvchi `x` yozsa — oxirgi False tarmoq **bo'lish**ni bajaradi! Chunki "+ emas, - emas, * emas" — bu hali "/" degani emas. Tuzatamiz.

3-bosqich: 2-versiya — xatolarni tekshirish

Ikkita yangi qoida: **bo'lishdan oldin** `b != 0` ni tekshir; **oxirgi** False tarmoqda "noma'lum amal" xabari. Natija hisoblanganini bilish uchun Boolean `xato` bayrog'i (flag) ishlatamiz:



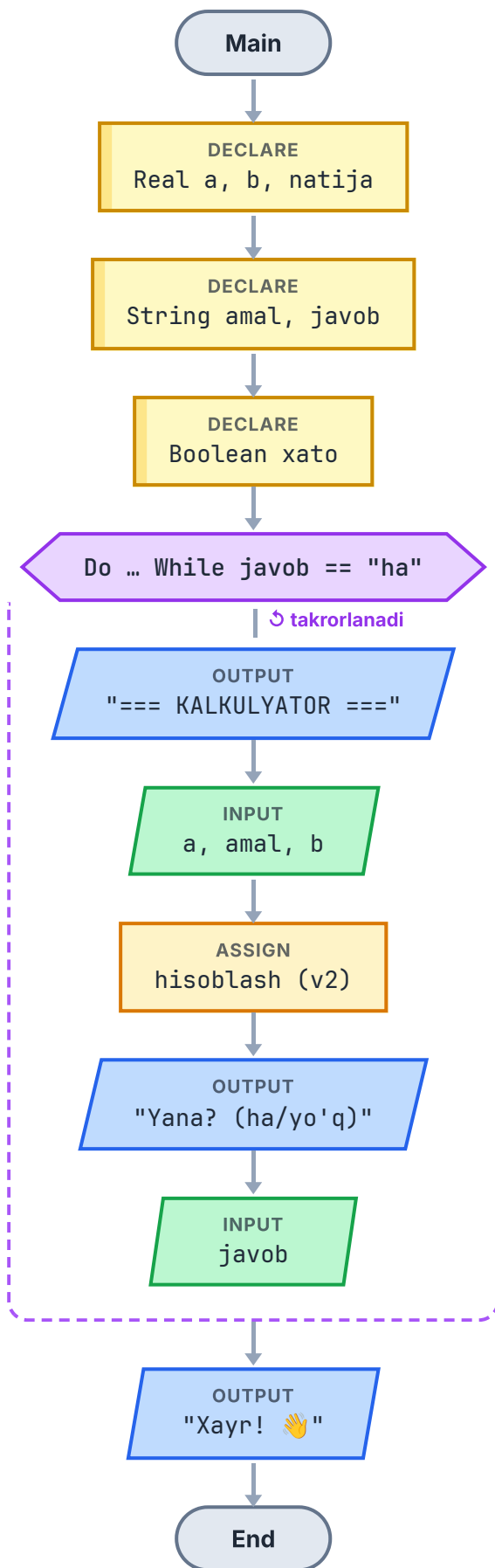
Bayroq (flag) usuli

Boolean o'zgaruvchi — "bayroq": boshida tushirilgan (`false`), muammo bo'lsa ko'tariladi (`true`). Oxirida bayroqqa qarab natijani chiqaramiz yoki chiqarmaymiz. Bu usul 23–25-loyihalarda ham kerak bo'ladi.

4-bosqich: 3-versiya — menyu va sikl

Haqiqiy kalkulyator bir hisobdan keyin yopilib qolmaydi. Dasturni **Do** sikliga o'raymiz: foydalanuvchi `"ha"` desa — davom etadi.

kalkulyator_v3.fprg — tashqi tuzilma



Console

```

=== KALKULYATOR ===
1-son:
15
Amal (+ - * /):
*
2-son:
4
15 * 4 = 60
Yana? (ha/yo'q)
ha
=== KALKULYATOR ===
1-son:
9
Amal (+ - * /):
/
2-son:
0
Nolga bo'lish mumkin emas!
Yana? (ha/yo'q)
yo'q
Xayr! 🖐️
  
```

Flowgorithm'da har bir Input bloki bitta o'zgaruvchi oladi — sxemada qisqartirib bitta blokda ko'rsatildi; dasturingizda `a`, `amal`, `b` uchun alohida Output + Input juftliklarini qo'ying.

To'liq Python versiyasi

Flowgorithm'ning Source Code Viewer'i bergan kodni soddalashtirsak, shunday bo'ladi:

kalkulyator.py

```
javob = "ha"
while javob == "ha":
    print("=== KALKULYATOR ===")
    a = float(input("1-son: "))
    amal = input("Amal (+ - * /): ")
    b = float(input("2-son: "))
    xato = False
    if amal == "+":
        natija = a + b
    elif amal == "-":
        natija = a - b
    elif amal == "*":
        natija = a * b
    elif amal == "/":
        if b == 0:
            print("Nolga bo'lish mumkin emas!")
            xato = True
        else:
            natija = a / b
    else:
        print("Noma'lum amal:", amal)
        xato = True
    if not xato:
        print(a, amal, b, "=", natija)
    javob = input("Yana? (ha/yo'q) ")
print("Xayr! 🖐️")
```

Brauzerda sinab ko'ring

Xuddi shu mantiq JavaScript'da — o'zingizning Flowgorithm natijalaringiz bilan solishtiring:

```

<div style="font:16px sans-serif;display:flex;gap:6px;flex-wrap:wrap;align-items:center">
  <input id="a" type="number" value="12"
style="width:80px;font:inherit;padding:6px">
  <select id="amal" style="font:inherit;padding:6px"><option>+</option>
<option>-</option><option>*</option><option>/</option><option>^</option>
<option>%</option></select>
  <input id="b" type="number" value="4"
style="width:80px;font:inherit;padding:6px">
  <button id="hisob" style="font:inherit;padding:6px 12px">=</button>
</div>
<p id="natija" style="font:20px monospace"></p>
<script>
document.getElementById('hisob').onclick = () => {
  const a = Number(document.getElementById('a').value), b =
Number(document.getElementById('b').value);
  const amal = document.getElementById('amal').value;
  let natija, xato = false;
  if (amal === '+') natija = a + b;
  else if (amal === '-') natija = a - b;
  else if (amal === '*') natija = a * b;
  else if (amal === '/') { if (b === 0) { xato = 'Nolga bo\'lish mumkin emas!';
} else natija = a / b; }
  else if (amal === '^') natija = a ** b;
  else if (amal === '%') { if (b === 0) xato = 'Nolga bo\'lish mumkin emas!';
else natija = a % b; }
  document.getElementById('natija').textContent = xato ? '⚠️ ' + xato : a + ' '
+ amal + ' ' + b + ' = ' + natija;
};
</script>

```

5-bosqich: test rejasi

#	a	amal	b	Kutilgan	Nimani tekshiradi
1	7	+	5	12	Qo'shish

#	a	amal	b	Kutilgan	Nimani tekshiradi
2	7	-	10	-3	Manfiy natija
3	2.5	*	4	10	Kasr son
4	7	/	2	3.5	Kasr natija
5	7	/	0	"Nolga bo'lish mumkin emas!"	Xato holat ⚠
6	0	/	7	0	0 ni bo'lish — mumkin!
7	7	x	2	"Noma'lum amal: x"	Noto'g'ri kiritish ⚠
8	—	—	—	"yo'q" → "Xayr!"	Sikldan chiqish

Qo'shimcha imkoniyatlar (yuqori ball uchun)

- **Daraja** (), **qoldiq** (, $b \neq 0$), **ildiz** (, $a \geq 0$ tekshiruvi bilan).
- Hisoblashlar **sonini** sanash va oxirida chiqarish: "Siz 5 ta amal bajardingiz".
- Barcha natijalar **yig'indisi** yoki **oxirgi natijani** keyingi hisobda sifatida ishlatish ("xotira" tugmasi).
- Amallarni raqamli menyu bilan tanlash:

Loyiha himoyasi

Mezon	Ball
4 amal to'g'ri ishlaydi	30
Nolga bo'lish va noto'g'ri amal tekshirilgan	25
Menyu sikli (qayta hisoblash)	15
Test jadvali (kamida 8 test)	15
Tushunarli xabarlar, GitHub'da README	10

Mezon	Ball
Qo'shimcha imkoniyat	5

Himoyada 2 daqiqa: **1)** loyiha maqsadi; **2)** sxemani ko'rsatib, mantiqni tushuntirish; **3)** jonli demo: oddiy holat + xato holat; **4)** qaysi qiyinchilik bo'ldi va qanday hal qildingiz.

Ko'p uchraydigan xatolar

- ✗ Oxirgi False tarmoqda shartsiz bo'lish → ✓ Har bir amalni aniq tekshiring; qolgani — "Noma'lum amal".
- ✗ `amal` ni Integer qilish → ✓ `+` — belgi, demak String.
- ✗ `amal == +` (qo'shtirnoqsiz) → ✓ `amal == "+"`.
- ✗ Xato bo'lganda ham eski `natija` ni chiqarish → ✓ Bayroq bilan tekshirish.
- ✗ Do sharti `javob == "Ha"`, foydalanuvchi `ha` yozdi → ✓ Aniq ko'rsatma: "(ha/yo'q)".

O'zingizni tekshiring

? Nolga bo'lish tekshiruvi qayerda turishi kerak?

- A. Dastur oxirida
- B. Input'dan oldin
- C. Bo'lish amalidan oldin
- D. Kerak emas

? Kalkulyatorni qayta-qayta ishlatish uchun qaysi sikl qulay?

- A. For 1 to 10
- B. Do ... While javob == "ha"
- C. Sikl kerak emas
- D. For 10 to 1

? 0 / 5 nima beradi?

- A. Xato
- B. 5
- C. Cheksizlik
- D. 0



Uyga vazifa

1. Kalkulyatorning 3-versiyasini yakunlab, kamida bitta qo'shimcha imkoniyat qo'shing.
2. Test jadvalini to'ldirib, skrinshotlar bilan GitHub'dagi `kaLkuLyator/README.md` ga joylang.
3. Sxemani **File** → **Export** → **Image** bilan rasmga olib, README'ga qo'shing.



Yodda tuting

Rejalashtirish

Maqsad, kirish, chiqish, qayta ishlash, xatolar — kod yozishdan oldin.

Tanlov

Zinapoya If'lar; oxirgi tarmoq — "noma'lum".

Himoya

Bo'lishdan oldin `b == 0` tekshiruvi.

Bayroq

Boolean `xato` : natijani chiqarish-chiqarmaslikni hal qiladi.

Menyu sikli

```
Do ... While javob == "ha".
```

☐ Darsni o'qib bo'ldim

✓ Test javoblari

1. Nolga bo'lish tekshiruvi qayerda turishi kerak? — **C) Bo'lish amalidan oldin**

Xato sodir bo'lishidan oldin tekshiramiz.

2. Kalkulyatorni qayta-qayta ishlatish uchun qaysi sikl qulay? — **B) Do ... While javob == "ha"**

Kamida bir marta hisoblash kerak, keyin foydalanuvchi qaror qiladi.

3. $0 / 5$ nima beradi? — **D) 0**

Nolni bo'lish mumkin; nolga bo'lish mumkin emas.